

SUPERVISED UNSUPERVISED LEARNING - Linear Regression Model

July 2, 2026

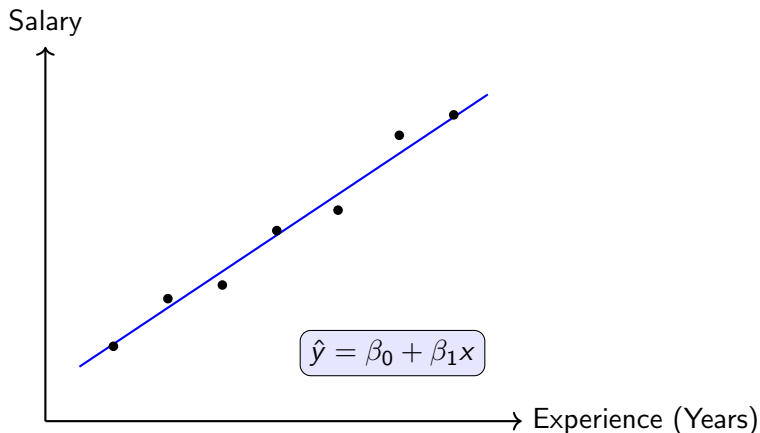
Session Outcomes

- Understand the concept of Linear Regression and explain its purpose in predicting continuous numerical values.
- Formulate a linear regression model by identifying dependent and independent variables.
- Estimate the regression coefficients (slope and intercept) using the least squares method.

What is a Linear Model?

- A **Linear Model** represents the relationship between the input features and the target using a straight line (2D) or a hyperplane (higher dimensions).
- Linear models are **parametric**, i.e., they have a fixed mathematical form with a small number of parameters that are learned from the training data.
- Due to their simple structure, linear models are:
 - Stable: small changes in the training data produce only small changes in the learned model.
 - Less prone to overfitting because they contain relatively few parameters.
 - Characterized by **low variance** but **high bias**.
- Linear models are well suited for small datasets where avoiding overfitting is important.
- They are widely used for
 - Regression
 - Classification

Linear Regression Model



$$\hat{y} = \beta_0 + \beta_1 x$$

where

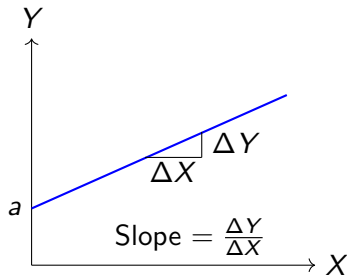
- x : Input feature & $\hat{y}$: Predicted output
- β_0 : Intercept (Bias) & β_1 : Slope (Weight)

Linear Regression Model

- The intercept determines the starting value of the line.
- The slope determines how rapidly the output changes with the input.

$$b = \frac{\Delta Y}{\Delta X}$$

(Change in Y per unit change in X .)



Prediction Error (Residual)

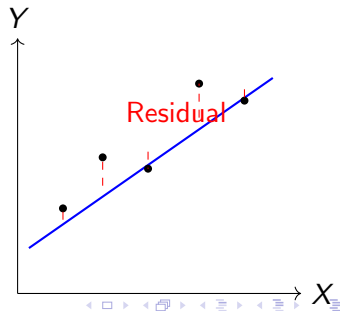
For every observation,

$$e_i = y_i - \hat{y}_i$$

where

- y_i : Actual value
- $\hat{y}_i$: Predicted value
- e_i : Residual (Prediction Error)

- Residual is the vertical distance between the actual point and the regression line.
- Smaller residuals indicate better predictions.
- Perfect prediction gives a residual of zero.



Simple Linear Regression

- Learn the relationship between an input feature (X) and an output variable (Y).
- Build a model that predicts the output for new unseen inputs.

Example Dataset

Weight (kg)	Height (cm)
74	170
80	180
75	175.5
78	177
82	182

Goal: New Weight $\longrightarrow$ Model $\longrightarrow$ Predicted Height

Observation

If there is only one input feature, the model is called **Simple Linear Regression**. If there are multiple input features, the model is called **Multiple Linear Regression**.

Linear Regression Model

The hypothesis (prediction) equation is

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

where

- θ_0 = Intercept (bias)
- θ_1 = Slope (coefficient)
- x = Independent variable (feature)
- $h_{\theta}(x)$ = Predicted output

Interpretation

- θ_0 decides where the line intersects the Y-axis.
- θ_1 tells how much the prediction changes for every one-unit increase in x .

For multiple input features,

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Cost Function (Mean Squared Error)

Predictions are rarely perfect. The error for each training example is

$$\text{Error} = h_{\theta}(x) - y$$

where

- $h_{\theta}(x)$ = Predicted value
- y = Actual value

The Mean Squared Error (MSE) cost function is

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

where

- 2 is added only to make differentiation easier
- m = Number of training samples
- $J(\theta)$ = Cost (average prediction error)

Goal:

Find θ_0, θ_1 such that $J(\theta_0, \theta_1)$ is minimum

Goal of Linear Regression

Goal/Aim: $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

$$\Rightarrow \min_{\theta_0, \theta_1} J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

How?

$\Rightarrow$ By continuously changing the values of θ_0 and θ_1 .

Example: Computing the Cost Function

Assume

$$\theta_0 = 0, \quad h_{\theta}(x) = \theta_1 x$$

Training data

x	y
1	1
2	2
3	3

Assume

$$\theta_1 = 0.5$$

Predictions become

$$h_{\theta}(1) = 0.5, \quad h_{\theta}(2) = 1, \quad h_{\theta}(3) = 1.5$$

Cost Calculation

Using

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x) - y)^2$$

Since $m=3$

$$\begin{aligned} J(\theta_1) &= \frac{1}{2 \times 3} [(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2] \\ &= \frac{1}{6} [0.25 + 1 + 2.25] \\ &= \frac{3.5}{6} = 0.5833 \end{aligned}$$

Interpretation

A large cost means the regression line does not fit the data well. Gradient Descent updates θ_0 and θ_1 to reduce this cost until the best-fit line is obtained.

Cost Function Example

$$h_{\theta}(x) = \theta_1 x$$

Let us assume $\theta_1 = 1$ (slope)

$$h_{\theta}(x) = x$$

$$x = 1 \Rightarrow h_{\theta}(x) = 1$$

$$x = 2 \Rightarrow h_{\theta}(x) = 2$$

$$x = 3 \Rightarrow h_{\theta}(x) = 3$$

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$J(\theta_1) = \frac{1}{2 \times 3} [(1 - 1)^2 + (2 - 2)^2 + (3 - 3)^2]$$

$$J(\theta_1) = 0$$

Cost Function Example for $\theta_1 = 2$

$$h_{\theta}(x) = \theta_1 x$$

Let us assume $\theta_1 = 2$ (slope)

$$h_{\theta}(x) = 2x$$

$$x = 1 \Rightarrow h_{\theta}(x) = 2(1) = 2$$

$$x = 2 \Rightarrow h_{\theta}(x) = 2(2) = 4$$

$$x = 3 \Rightarrow h_{\theta}(x) = 2(3) = 6$$

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

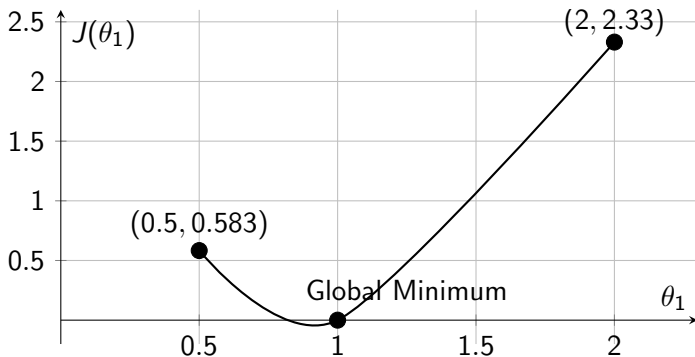
$$J(\theta_1) = \frac{1}{2 \times 3} \left[(2 - 1)^2 + (4 - 2)^2 + (6 - 3)^2 \right]$$

$$J(\theta_1) = \frac{1}{6} \left[1^2 + 2^2 + 3^2 \right]$$

$$J(\theta_1) = \frac{1}{6} [1 + 4 + 9] = \frac{14}{6} = \frac{7}{3}$$

Cost Function Curve for Different θ_1 Values

θ_1	$J(\theta_1)$
0.5	0.583
1.0	0
2.0	2.33



Convergence Algorithm

Convergence Algorithm The main aim of gradient descent is to optimize the change of θ values and achieve the global minimum.

Repeat until convergence

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_j)$$

where,

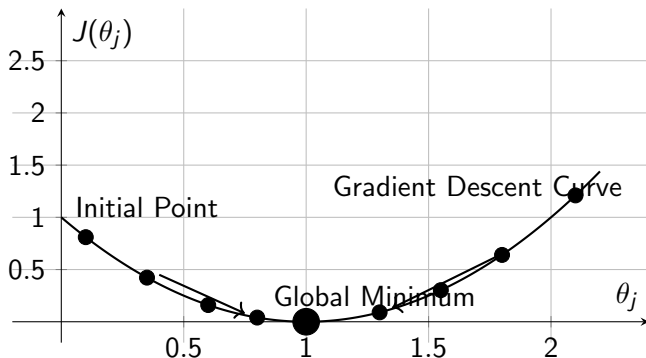
α = learning rate

Here, α controls the step size. If α is very large, the algorithm may jump over the minimum. If α is very small, the algorithm will move very slowly

$$\frac{\partial}{\partial \theta_j} J(\theta_j) = \text{slope of the cost function}$$

Main aim: $J(\theta_j) \rightarrow \text{Global Minimum}$

Gradient Descent Curve



$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_j)$$

Gradient Descent is an optimization algorithm that repeatedly updates the model parameters in the direction that reduces the cost function until the minimum error (global minimum) is reached.

How to Compute the Slope?

To update θ_j , first we calculate the slope of the cost function.

$$\frac{\partial}{\partial \theta_j} J(\theta_j)$$

This slope tells us the direction in which the cost function is changing.

Case 1: Negative slope

If the slope is negative,

$$\frac{\partial}{\partial \theta_j} J(\theta_j) < 0$$

then,

$$\theta_j := \theta_j - \alpha(-ve)$$

$$\theta_j := \theta_j + \alpha(positive)$$

So, θ_j increases and moves towards the minimum.

Explanation

Gradient descent is an optimization algorithm.
Its main aim is to reduce the cost function value step by step.

$$J(\theta_j)$$

The parameter θ_j is updated repeatedly using:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_j)$$

At the global minimum,

$$\frac{\partial}{\partial \theta_j} J(\theta_j) = 0$$

So the cost function reaches its lowest value.

Derivative with Respect to θ_0

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

For $j = 0$:

$$\begin{aligned} \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) &= \frac{\partial}{\partial \theta_0} \left[\frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \right] \\ &= \frac{1}{2m} \sum_{i=1}^m 2 \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \frac{\partial}{\partial \theta_0} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \\ &= \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \end{aligned}$$

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)$$

Gradient Descent Update Rules

For $j = 1$:

$$\begin{aligned}\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} &= \frac{\partial}{\partial \theta_1} \left[\frac{1}{2m} \sum_{i=1}^m \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2 \right] \\ &= \frac{1}{m} \sum_{i=1}^m \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right) x^{(i)}\end{aligned}$$

Since,

$$h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$$

we can write:

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

Gradient Descent Algorithm

Repeat until convergence

$$\begin{cases} \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \\ \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)} \end{cases}$$

where,

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

α = learning rate